

Choix des technologies

1. Langages et environnement d'exécution

Python 3.11

Python a été retenu comme langage principal du projet en raison de son écosystème riche pour la data, le machine learning, le développement d'API et l'automatisation. Il permet de couvrir l'ensemble de la chaîne de traitement, depuis l'ingestion des données jusqu'à l'exposition des résultats via des services web.

R

R a été utilisé pour les phases de préparation et de nettoyage des données. Ses fonctions natives permettent de traiter efficacement les valeurs manquantes, les formats de dates et les incohérences de données avant leur intégration dans les analyses.

2. Versionnage et collaboration

Git et GitHub

Nous avons utilisé Git pour assurer le suivi des modifications du code et garantir la traçabilité des contributions de chaque membre de l'équipe. GitHub a servi de plateforme collaborative permettant le partage du code, la gestion des branches et l'intégration des différents volets du projet.

3. Manipulation et analyse des données

Pandas

Pandas constitue la bibliothèque principale de manipulation de données. Elle a été utilisée pour le chargement, le nettoyage, la transformation et l'agrégation des relevés de consommation, des données météorologiques et des informations clients.

NumPy

NumPy fournit les structures de données et opérations numériques nécessaires au traitement de grands volumes de données.

SciPy

SciPy a été utilisé pour certaines opérations statistiques avancées et pour l'analyse exploratoire des données.

4. Machine Learning et analyse prédictive

Scikit-learn

Scikit-learn a été retenu pour sa simplicité d'utilisation et sa robustesse.

Les algorithmes suivants ont été utilisés :

- Isolation Forest pour la détection automatique d'anomalies et de fraudes potentielles sur les compteurs.
- K-Means pour segmenter les clients selon leurs profils de consommation.
- TF-IDF pour analyser les réclamations clients et identifier les thèmes récurrents.

Statsmodels

Cette bibliothèque a été utilisée pour les analyses statistiques et l'étude des séries temporelles de consommation énergétique.

Joblib

Joblib permet la sauvegarde et le rechargement des modèles entraînés afin d'assurer leur réutilisation et leur déploiement futur.

5. Visualisation et aide à la décision

Plotly

Plotly a été utilisé pour créer des tableaux de bord interactifs facilitant l'exploration des données par les utilisateurs métiers.

Matplotlib et Seaborn

Ces bibliothèques ont permis la réalisation de graphiques d'analyse exploratoire et de visualisations statistiques détaillées.

Power BI

Power BI a été retenu pour la restitution des résultats auprès des décideurs. Il permet la création de tableaux de bord dynamiques adaptés aux besoins des équipes d'exploitation, de la direction financière et du service client.

6. API et services applicatifs

FastAPI

FastAPI a été choisi pour développer rapidement une API REST performante permettant l'accès aux données, aux indicateurs et aux résultats des modèles.

Uvicorn

Uvicorn assure l'exécution de l'application FastAPI avec de bonnes performances et une faible consommation de ressources.

Pydantic

Pydantic garantit la validation et la cohérence des données échangées entre les différents composants de la plateforme.

7. Stockage des données

SQLite

SQLite a été utilisée comme base de données du prototype afin de simplifier le déploiement et les tests pendant la phase de développement.

PostgreSQL

PostgreSQL est identifié comme solution cible pour une mise en production. Cette base relationnelle offre de meilleures performances, une meilleure gestion de la concurrence et une plus grande capacité de montée en charge.

8. Conteneurisation et déploiement

Docker

Docker permet d'assurer la reproductibilité des environnements et de simplifier le déploiement de l'application sur différents systèmes.

Docker Compose

Docker Compose facilite l'orchestration des différents services du projet (API, base de données, outils de traitement).

9. Cybersécurité et DevSecOps

Pip-Audit

Pip-Audit a été utilisé pour détecter les vulnérabilités connues dans les dépendances Python et renforcer la sécurité de l'application.

EBIOS Risk Manager

La méthode EBIOS RM a servi à identifier les risques liés à l'infrastructure critique de Néovolt et à définir des mesures de protection adaptées.

SIEM

Une logique de supervision inspirée des solutions SIEM a été mise en place pour centraliser et analyser les événements de sécurité.

OWASP ZAP et Nmap

Ces outils ont été utilisés dans un cadre contrôlé afin d'évaluer la surface d'attaque du prototype et d'identifier d'éventuelles failles de sécurité.

10. Documentation et industrialisation

Markdown

Markdown a été utilisé pour produire une documentation claire, versionnable et facilement maintenable.

XHTML2PDF, ReportLab et Pandoc

Ces outils permettent la génération automatisée de documents PDF à partir des rapports produits par les différents volets du projet.

Mermaid

Mermaid a été utilisé pour générer les schémas d'architecture, les diagrammes de flux et les représentations techniques du système.

11. Concepts mobilisés

Le projet s'appuie sur plusieurs concepts clés :

- Détection d'anomalies non supervisée ;
- Segmentation client par clustering ;
- Analyse de texte par NLP ;
- Analyse de séries temporelles ;
- Principes MLOps ;
- Gouvernance et protection des données ;
- Conformité RGPD et NIS 2 ;
- Analyse d'impact (AIPD) ;
- Sécurité de l'information selon les principes ISO 27001.

Ces choix technologiques répondent aux exigences de performance, de sécurité, d'évolutivité et de gouvernance attendues pour une infrastructure énergétique critique telle que Néovolt.